CodebookOS

V1.0 MANIFESTO

A 25.4 KB bare-metal x86_64 OS with a custom programming language.
Pure NASM UEFI. No borrowed code.

Substrate size:	25.4 KB hand-crafted x86_64 NASM UEFI
Architectural doctrines:	44 codified through V1.0 SEAL
Canary-verified demos:	6 CBS programs at byte-exact precision
Build effort:	30 architect-hours, solo, April-May 2026
V1.0 SEAL contract sha:	c9923b8cf9fb6caf4c195e2d0d0ea2ed...
Demo video:	90.000000s; h264; 1280x720

Randolph Pelican III / StableTech Enterprises LLC
May 2026
github.com/RandolphPelican/codebook

1. What CodebookOS is

CodebookOS is a bare-metal operating system with its own programming language, built from scratch in 25.4 KB of hand-written x86_64 NASM UEFI assembly. It boots in QEMU; it flashes to USB; it runs six byte-exact-verified CBS demonstration programs against five typed primitive pools. Every architectural decision is codified as one of 44 doctrines preserved in the repo. The substrate is auditable in a fortnight by a competent reviewer.

The substrate is the credential. This document is the depth-doc tour of what it is, how it was built, and what discipline holds it together.

Headline anchors

- 25.4 KB of non-zero substrate bytes in BOOTX64.EFI (post-stripping)
- 44 codified architectural doctrines through V1.0 SEAL, plus 8 D4.X doctrines through V1.0 SHIP
- 6 canary-verified CBS programs covering the full Maid V1.0 capability surface
- Two-build determinism preserved across 16+ substrate-evolution pods
- F32 IEEE 754 byte-exact reproducibility per Form A canonical evaluation order
- Built solo in 30 architect-hours across 3 months (April-May 2026)
- V1.0 SEAL contract sha: c9923b8cf9fb6caf4c195e2d0d0ea2ed4a8e51e4e9f827b1fc24dd0b28c1d900

What this manifesto is for

This document is the fortnight-auditor's companion. It is not a tutorial; the GETTING_STARTED guide in the repo is the tutorial. It is not a language reference; CBS_LANGUAGE.md is the reference. This document explains why the substrate looks the way it does, what discipline produced it, and what reviewers should look for when they read the source.

2. The substrate - five typed primitives

Every value in CBS that has identity is one of five types. Each lives in a pre-allocated pool with bounded capacity. Each has a SipHash-2-4 MAC guarding integrity where applicable. Each has its own opcode row in the dispatch table. These five primitives are the substrate.

Sign - declarations of intent

A Sign is the substrate's first-class declaration object. It carries an Energy reference (metabolic budget for actions taken under this Sign), an embedding_handle pointing into the Embedding pool (the "meaning" of the Sign), and an owner cap_id with arena (provenance per D1.10.2b2.1). Pool capacity is 256 slots after the D3.16 anticipated-empirical-pressure expansion.

Energy - metabolic budget

Energy is non-renewable spending capacity for opcodes. Every opcode declares its cost in joules; the VM decrements the active budget register (r14) at each dispatch; depletion triggers graceful HALT. The substrate cannot execute beyond a cap's budget. Per D3.17, costs are anticipated-worst-case static prices rather than measured machine work - the substrate trades cost-table precision for never-undershooting.

Outcome - Ok<T> or Err

The substrate's error-handling primitive - a tagged union over typed Ok payloads (with value_type_id discriminant) and standardized 32-byte error contexts. Every multi-result operation in V1.0 returns Outcome. Pop the Outcome, branch on outcome_is_ok, unwrap accordingly. There is no exception system, no error-by-side-channel.

Cap - capability tokens

Cap is the authority primitive. Every action that touches restricted resources checks the active cap's bitmap and energy budget. Caps form a tree rooted at ROOT_CAP (cap_id=1, unbounded). Each child cap is forged via cap_new with subset-on-grant semantics (D2.2.5): cap_new cannot grant a bit the parent does not hold. The cap framework is enforced from layer 1 - no opcode bypasses it.

Embedding - vector representations

Embedding is the semantic primitive: a 384-dimensional f32 vector (matching all-MiniLM-L6-v2 dimensionality) with a SipHash MAC over the full vector body (D3.3). Mutate one f32, the MAC breaks. This is the substrate's interface to high-dimensional meaning - cosine similarity, geometric projection, codebook lookup. The Maid V1.0 surface exposes 6 capability variants over this primitive.

The Maid V1.0 surface - six capabilities

Pod	Surface	Capabilities
3.5	Housekeeper	cosine + dot + L2 + lookup_top1 + sign_handle
3.6	Composer	add + subtract + scale + normalize + lerp + synthesis_handle
3.8	Importer	boot_ingest_codebook + imported_handle
3.9	Finder-of-many	lookup_top_k with threshold

3.10

Orthogonalizer

project + reject

3.11

Maintainer

codebook_meta

The Maid is the lexical-computation pole of the substrate's planned cognitive trinity. V1.0 ships one of three pillars complete. The other two - Cop (capability inspector) and Interpreter (text-to-bytecode runtime translation) - carry forward to V2.0 per the D3.43 V1.0-deferral framework.

3. The language - CBS

CBS (Custom Bytecode Substrate) is the substrate's programming language. Custom syntax, custom compiler (tools/atreyu_x86.py, ~4,200 lines of Python), custom stack-VM (boot/cbs_vm.asm, ~3,900 lines of NASM). Every opcode is energy-accounted; every primitive operation is byte-exact reproducible.

Authoring model at V1.0

CBS programs at V1.0 are authored as Python AST functions in tools/atreyu_x86.py. Each demo function returns an AST tree which the compiler walks to emit bytecode. The compiled .cbc file is what runs on the substrate. The substrate also includes a self-hosted parser chain for textual CBS source (surfaces/parser.cbs + surfaces/lexer.cbs), but the AST-based authoring path is the canonical credential-tier path used by all 6 canary demos.

Capability-tokenized I/O (D4.2)

Every CBS interaction with substrate services goes through OP_USE_CAP(token, cmd) with one of four V1.0 capability tokens: AURYN_DISPLAY (framebuffer), GMORK_CONIN (keyboard), MORLA_FS (filesystem), ROCKBITER (energy introspection). The dispatch table for capabilities is a flat enum keyed by token. New surfaces in V2.0 add new tokens without new opcodes.

Per-opcode cost table (excerpt)

```
0x01 OP_PUSH                     1j     Push i64 literal
0x10-0x13 OP_ADD/SUB/MUL/DIV   1-3j    i64 arithmetic
0x70/0x71 OP_LOAD/OP_STORE       1j     Variable load/store
0x80 OP_PRINT_NUM                2j     Print integer (I/O)
0x91 OP_USE_CAP                  1j     Capability service dispatch
0xA0 OP_SIGN_NEW               100j     Forge typed Sign primitive
0xB0 OP_CAP_NEW                100j     Forge typed Cap primitive
0xC0 OP_EMBEDDING_NEW          100j     Forge typed Embedding primitive
0xC6 OP_EMBEDDING_COSINE       400j     f32 cosine (D3.14 Form A)
0xC9 OP_EMBEDDING_LOOKUP_TOP1  100000j Pool-bounded scan (D3.17)
0xCA OP_EMBEDDING_ADD          500j     f32 vector add (D3.6)
0xCD OP_EMBEDDING_NORMALIZE    700j     With zero-norm rejection
0xF2 OP_EMBEDDING_LOOKUP_TOP_K 100000j Top-K cosine ranking (D3.35)
0xF3 OP_EMBEDDING_PROJECT     1500j     f32 geometric project (D3.38)
0xF4 OP_EMBEDDING_REJECT      1500j     f32 geometric reject (D3.38)
0xFF OP_HALT                     0j     Termination
```

All values are anticipated worst-case per D3.17 - not measured machine work. The substrate's r14 register enforces budget, not cost-accuracy. The substrate prefers fixed pricing decisions over per-pod cost-table re-tuning.

4. Walked demonstration - the credential in action

The six V1.0 canary demos exercise the full Maid surface plus all four capability tokens plus Outcome unwrap plus the cap lifecycle. Three walked examples follow.

B53: Fibonacci with energy trace

A CBS program computes fib(12) iteratively and prints the running energy budget at each iteration. The substrate's per-opcode cost table makes energy accounting visible per iteration. Sample output:

```
fib(0) = 0
fib(1) = 1
fib(2) = 1   joules used: 87
fib(3) = 2   joules used: 115
fib(4) = 3   joules used: 151
...
fib(12) = 144   joules used: 407
Energy: 445j used, 999555j remaining
```

D3.17 anticipated-worst-case empirical at user-program scale. Every cycle of the iteration adds 28-36 joules; the costs accumulate predictably and the budget enforcement is observable.

B55: Vector composer - five-doctrine cross-composition

A four-step f32 vector composition chain demonstrates substrate-level cross-doctrine composability: ADD, then SCALE by 0.5, then PROJECT onto axis vector, then REJECT against orthogonal vector. The halving-magnitudes cascade goes 2.0 -> 0.5 -> 0.25 -> 0.125. The final dot product of the rejected vector with the orthogonal vector is byte-exact 0.0 per the D3.40 clean-cancellation regime of the hybrid IEEE-degeneracy convention. 13 byte-exact predictions match per the B55 canary; energy consumption is 5,647 joules for the full chain.

B56: Capability lifecycle

Origin -> grant -> use -> accounting. The CBS program observes ROOT_CAP (cap_id=1, unbounded), forges a subcap via cap_new with subset-on-grant semantics (D2.2.5), enters the subcap context via cap_enter, performs operations, exits back to ROOT_CAP. The demo honestly documents what V1.0 ships (grant + use + lineage) versus what V2.0 carries forward (cap_revoke, federation_total ripple, spatial-merge ripple). Real substrate discipline, not aspirational design.

The other three demos

- B57 press-X: first V1.0 interactive CBS program; polls use_cap(CAP_GMORK_CONIN, 1) at ~29 joules per poll; ~145,080 joules consumed across 5,000 polls for a typical "press X to continue" interaction.
- B54 similarity browser: Maid top-K cosine ranking against the boot-time-ingested codebook; CBKBOK01 format (D3.30); per-embedding provenance via imported_handle (D3.8).
- B58 drift anchor: D3.28 self-verifying canon; renders 0xB4000000 framebuffer drift byte-exact at runtime as substrate-canon witness.

5. The doctrinal corpus - 44 + 8 codified decisions

Every architectural decision in CodebookOS lands as a doctrine in a pod's decision record. Decisions are codified before implementation when possible (HALT 1 pattern), empirically verified at canary stage, numbered globally so cross-pod references work mechanically, and cited at use in code comments.

D1.X - substrate plumbing era

- D1.9.1.1 - Tagged Outcome with value_type_id discriminant. Foundation for every error path that follows.
- D1.10.1.1 - Cap slot layout (128-byte symmetric, no mirror fields). Drives MAC signature, parent walks, bitmap checks.
- D1.10.1.7 - SipHash-2-4 over 6 u64 fields. Universal MAC convention; all four MAC-bearing primitives use this signature.
- D1.10.2a.2 - RDSEED -> RDRAND -> hard-fail-and-halt policy. Substrate-secret bootstrap; refuse to boot rather than ship a fixed secret.
- D1.10.2b1.9 - Substrate witnesses its own authority context for the first time. cap_current, cap_arena, cap_owner activate the dormant arena/owner fields in earlier primitives.

D2.X - Babylon spatial-merge era

- D2.1.4 - ROOT_CAP accumulates federation total. Every Outcome forge anywhere ripples energy_used up the cap tree. Substrate-wide accounting in one place.
- D2.2.1 - cap_bitmap structured semantics: texture as physics. Each bit is a granted-or-denied capability.
- D2.2.5 - Subset-on-grant capability-correctness invariant. cap_new cannot grant a bit the parent does not hold. Standard cap-system invariant; enforced at forge.

D3.X - Embedding + Maid V1.0 era

- D3.1 - Embedding as fifth typed primitive.
- D3.2 - Canonical V1.0 dimension EMBEDDING_DIM = 384.
- D3.3 - Full vector under MAC protection.
- D3.12 - FP determinism doctrine: SSE-scalar single-precision only. No x87 80-bit; no AVX2 reorderings under user control; movss/mulss/addss only. Foundation of F32 byte-exact determinism.
- D3.14 - Cosine canonical Form A; bit-exact load-bearing. The order of accumulation in dot-product reductions is fixed. Same vector -> same f32 bits, every run.
- D3.17 - Static worst-case costing for compute composites. Substrate prefers fixed pricing over per-pod re-tuning.
- D3.25 - Forge-tier introduction; Maid as lexical-computation pole; Trinity-naming canonization.
- D3.28 - The project learns how to learn from its FP frontier. Form A discipline + hybrid IEEE-degeneracy convention + canary self-verification.
- D3.30 - CBKBOK01 codebook image format. Boot-time ingestion of embedding sets; substrate-private 0j operation.
- D3.37 - NASM RIP-relative indexed-BSS-access discipline. Substrate-catch landing from a six-probe diagnostic chain in Pod 3.9. lea reg, [rel sym]; [reg + idx*scale]; never [rel sym + reg*scale].

- D3.38 - Project-Reject duality as orthogonalization primitive pair.
- D3.40 - Hybrid IEEE-degeneracy convention extension. Zero-norm vectors and clean-cancellation regimes both fold to byte-exact 0.0.
- D3.43 - V1.0-deferral framework (broad). Carries V2.0 work-items through framework tests at activation time; honest "not yet" rather than vague "future work."
- D3.44 - Catch-surface-migration tri-tier doctrine. Catches above expected tier are signal; catches at-or-below expected tier are noise.

D4.X - V1.0 SHIP polish-layer era

- D4.1 - Polish-vs-credential separation. boot/ + surfaces/ + tools/ = credential; polish/ = showroom. The substrate sha must not change during polish work - empirically verified across nine consecutive Pod 4.0 chunks.
- D4.2 - Capability-tokenized I/O surface. Every CBS interaction with substrate services goes through OP_USE_CAP with one of 4 tokens.
- D4.3 - Boot animation discipline.
- D4.4 - In-fiction surface discipline. Surfaces that don't ship at V1.0 get polish-layer mocks for the demo video; the mock is honestly framed as in-fiction.
- D4.5 - Demo-program discipline.
- D4.6 - Release-artifact discipline (this pod).
- D4.7 - Public-repo-flip discipline (next pod).
- D4.8 - Polish-layer verification discipline. Tier 1 = byte-exact (substrate canaries). Tier 2 = output-existence + format-sanity + sampled-decode (polish artifacts).

The doctrine corpus is the substrate's audit trail. A reviewer reading recon/ in chronological order sees every architectural decision in the order it was made, the alternatives considered, and the empirical evidence that ratified it. The 30-architect-hour buildout is reproducible in concept because every decision is preserved.

6. The build methodology - chunked pods

Every architectural change in CodebookOS goes through a pod: a unit of bounded scope, codified upfront, executed in chunks, sealed with a decision record. The skeleton:

Recon (chunk A)

Read the relevant existing source and prior decision records. Identify the design space. Produce a recon report. End recon with questions for the architect - never start implementation while design ambiguity remains.

HALT 1 - architectural ratification

Architect reviews recon plus questions. Decisions made are codified as doctrines (D<phase>.<num>) before any code is written. HALT 1 closes when every question has a ratified answer; the doctrines that will land are named upfront.

Execution chunks (B-N)

Each chunk is bounded (typically 1-3 hours of architect attention). Substrate edits happen in chunk-bounded slices. Each chunk closes by running a canary that verifies the chunk's promises. Substrate sha is verified at every chunk close (two-build determinism).

Canary verification

The substrate canary boots the OS in QEMU, runs a target CBS demo, captures framebuffer, exits. Pre-canary hash to post-canary hash comparison detects any drift. Auxiliary substrate canaries wrap a temporary substrate change (e.g., a codebook ingestion) for a single canary run, then revert.

SEAL (final chunk)

Pod's decision record lands every doctrine plus catch profile plus state at SEAL. Three-oracle verification: git rev-parse HEAD = git rev-parse origin/main = git ls-remote origin main. Commit message follows the pod-prefix format. Substrate sha invariant verified one more time across the SEAL commit.

The methodology trades velocity for discipline. A pod takes 1-3 days of architect attention; that yields a substrate change with a codified rationale, byte-exact verification, and a paper trail.

7. Empirical verification - what was actually measured

Two-build determinism

Assembling boot/boot.asm twice with the same NASM version produces byte-exact identical BOOTX64.EFI. The build script verifies the V1.0 SEAL contract sha at every build. Verified across 16 substrate-pod chunks (Pod 3.0 through Pod 3.12 SEAL), then frozen at V1.0 SEAL contract c9923b8cf9fb6caf4c195e2d0d0ea2ed4a8e51e4e9f827b1fc24dd0b28c1d900. The D4.1 byte-lock extends this guarantee through V1.0 SHIP: no polish-tier work has touched substrate bytes across 9 consecutive Pod 4.0 chunks.

F32 IEEE 754 byte-exact determinism

Every f32 op in the Maid V1.0 surface uses Form A canonical evaluation order (D3.14). The same input vector produces the same f32 bit pattern across runs, builds, and architectures when ported. Verified per canary: B53 fib energy trace, B58 drift anchor, B55 vector composer all rely on byte-exact f32 results.

Pool sizing - D3.29 axis-2 mechanical proportionality

Pool	V1.0 capacity	MAC-protected
Sign	256 slots	No (V1.0; future via OP_SIGN_PROV)
Energy	256 slots	No (V1.0; non-MAC accessor)
Outcome	4096 slots	Yes (D3.29 proportional sizing)
Cap	256 slots	Yes (SipHash MAC over 6 fields)
Embedding	2048 slots	Yes (SipHash MAC over 384 f32 = 196 qwords)

Cost-table empirical anchoring

The B53 fibonacci canary makes the cost table empirically observable. Each iteration of the iterative fib(n) loop consumes ~28-36 joules; the substrate's r14 register decrements as documented in the per-opcode cost table. D3.17 anticipated-worst-case pricing yields predictable accumulation; cost-table internal consistency is verified at the composition layer across the B55 vector composer chain.

8. The polish layer - D4.1 separation

CodebookOS has two disciplines, separated by directory and by purpose.

Credential tier

boot/ + surfaces/ + tools/ + recon/. Pure substrate. Touches the substrate sha. Requires three-oracle verification, canary byte-exact, doctrine-grade rationale for every change. This is what is being claimed as the credential.

Polish tier

polish/. Python only. Cannot import from boot/. Cannot regenerate .cbc files. Cannot affect the substrate sha. Boot animation, About demo, in-fiction surface mocks, demo video composition pipeline, and the depth-doc PDF builder (this manifesto) all live here. This is the showroom that makes the credential visible.

The D4.1 byte-lock - empirically established

Through V1.0 SHIP, the substrate sha at V1.0 SEAL has remained unchanged across nine consecutive Pod 4.0 chunks (Pod 4.0.C through Pod 4.0.H): c9923b8cf9fb6caf4c195e2d0d0ea2ed.... The polish layer can be deleted entirely and the substrate still builds, boots, and passes all canaries. This separation is what allows V1.0 to ship with a polished demo video without compromising the credential's purity.

9. V1.0 SEAL - honest scope

CodebookOS V1.0 ships exactly what is built. Aspirational features carry forward as V2.0 candidates framework-tested per D3.43 at activation time. Honest "not yet" rather than vague "future work."

Surface status at V1.0

Surface	V1.0 status	V2.0 carry-forward
Substrate (5 typed pools)	Complete	-
Maid V1.0 (6 capabilities)	Complete	-
CBS language + compiler + VM	Complete	-
Capability framework (grant/use/lineage)	Complete	cap_revoke; federation_total; spatial-merge ripple
Codebook ingestion (boot-time + read)	Complete	Runtime IMPORT (#91); multi-codebook
Stream-stability / aggregation ops	Deferred	#92 - Result[T] sixth pool if production demand
Cop (capability inspector)	Deferred	Trinity pillar 2
Interpreter (text-to-bytecode runtime)	Deferred	Trinity pillar 3
Demod-tier surface (0xE8-0xEF reserved)	Deferred	V2.0
Falkor / Atreyu / Rockbiter as live	Deferred (in-fiction mocks)	V2.0

What V1.0 is NOT

- Not a general-purpose OS. No process scheduler, no virtual memory, no syscall interface for user programs beyond the capability-tokenized I/O surface.
- Not a networked system. No TCP/IP, no Ethernet driver. The substrate runs entirely on the bare metal that boots it.
- Not a multi-user system. Single-user, single-active-cap-context (with cap_stack for nested authority). User authentication is deferred.
- Not a self-hosted development environment. CBS demos compile on a host (Linux/macOS/WSL2) with Python; the substrate runs them but does not compile them at runtime. Runtime IMPORT is V2.0 (#91).

What V1.0 IS

25.4 KB of hand-written NASM that boots in QEMU, runs 6 byte-exact CBS demonstration programs against 5 typed primitive pools, with 44 doctrines codifying every architectural decision and 8 D4.X doctrines codifying the V1.0 SHIP polish discipline. The trinity has one pillar complete. The next two pillars will be built on the same substrate.

10. The mythology

The substrate's named surfaces honor The Neverending Story (Michael Ende, 1979). Mythology naming is load-bearing: when a Gmork command says "auryn" or "morla", the architect-team can locate the responsible NASM file at a glance, and the API discipline (capability tokens, doctrine annotations) flows from the name.

Name	Role in CodebookOS
Bastian	Home screen - the boy who reads
Atreyu	CBS programming language - the warrior on the quest
Falkor	Web browser surface (V2.0; polish-layer mock at V1.0) - the luckdragon
Gmork	Terminal shell - interpreter of the dark
Auryn	Display / framebuffer - the amulet that protects
Morla	Filesystem - the ancient turtle who knows
Rockbiter	Energy introspection - "good strong stones"
Koreander	Bookmaster - codebook ingest at boot
Empress	Capability framework - ROOT_CAP and cap_stack
Babylon	Spatial-merge metabolism - federation accounting
Maid	Lexical-computation pole - 6 capabilities live at V1.0
Cop	Capability inspector - V2.0 trinity pillar 2

The mythology is fair-use literary reference; no commercial relationship with the Michael Ende estate is implied or claimed.

11. What was learned

A 30-architect-hour project across three months produces a particular kind of artifact: small enough to audit, disciplined enough to extend, opinionated enough to teach. The lessons that survived the substrate-evolution sequence:

Codify decisions before implementing them

The HALT 1 pattern - recon + questions + architect ratification + doctrine landing before any code - saved time in every pod where it was followed. The substrate-catches that landed empirically (D3.37 NASM RIP-relative, D3.41 forge-id literal discipline) happened in pods where the design space was clear but the implementation language was less forgiving than expected. The doctrines that landed pre-implementation never had to be reversed.

Anticipated worst-case is better than measured average

D3.17 (static worst-case costing) was the right call. Per-pod cost-table re-tuning would have multiplied the audit surface; fixed conservative pricing keeps the cost table small, the substrate predictable, and the user-program budget never undershooting. Empirical observability (B53 fib trace) emerges naturally from anticipated worst-case pricing.

Mythology naming is a real engineering tool

Naming Atreyu the compiler and Maid the computation pole did more than make the repo memorable. It gave each architectural surface a metonym that the architect-team could use in design discussion without ambiguity. "Add a Maid capability" is unambiguous; "add a vector-op capability" is not. The mythology naming is not flavor; it is engineering scaffolding.

Two-build determinism is cheap and load-bearing

Verifying byte-exact reproducibility at every SEAL commit is a tiny additional cost (one extra `nasm + sha256sum`). It catches latent nondeterminism early and certifies that the substrate sha is a meaningful contract. Without it, the V1.0 SEAL contract c9923b8cf9fb6caf4c195e2d0d0ea2ed... would be aspirational rather than empirical.

Polish-vs-credential separation must be empirical, not stated

D4.1 byte-lock is not a rule on a wiki page. It is verified at every Pod 4.0 chunk close: the substrate sha after polish-tier work must equal the substrate sha before polish-tier work. Nine consecutive chunks have honored this discipline. The separation is real because it is measured.

12. Repository reference - what to read in what order

For a competent reviewer doing a fortnight audit:

- README.md - 5-minute orientation. Front door for any reader.
- GETTING_STARTED.md - 10-minute hands-on: clone -> build -> boot -> Gmork -> demo.
- boot/boot.asm - UEFI entry, PE32+ header, boot initialization (~500 lines).
- boot/defines.asm - opcode constants, capability tokens, pool sizes. The substrate's grammar in one file.
- boot/cbs_vm.asm - stack-VM dispatch + per-opcode handlers (~3,900 lines). The substrate's execution core.
- boot/cap.asm - capability framework: cap_new, cap_enter, cap_exit, MAC verification, parent walks.
- boot/maid.asm - Maid V1.0 compute helpers (~700 lines). The lexical-computation pole's f32 substrate.
- recon/POD3_DECISION_RECORD.md - Embedding primitive landing.
- recon/POD3.12_DECISION_RECORD.md - V1.0 SEAL pod; deferral framework + catch-surface-migration doctrine landing.
- tools/atreyu_x86.py - CBS compiler (~4,200 lines). Full language definition in one Python file.
- 6 canary demos: surfaces/test_pod40f_b53..b58.cbc + their tools/atreyu_x86.py:demo_pod40f_b5X source functions.
- CBS_LANGUAGE.md - the language reference.
- ARCHITECTURE.md - the doctrinal depth tour.
- CONTRIBUTING.md - the pod methodology + style notes.

Read recon/ in chronological order to see how the substrate evolved decision by decision. The mythology naming helps - every NASM file and every architectural surface has a name you can keep straight while you read.

Closing

The substrate is the credential. The substrate's discipline is the doctrine corpus. The doctrine corpus is the audit trail. This manifesto is the depth-doc tour, but the real document is the repository - 25.4 KB of NASM plus the recon/ folder.

CodebookOS V1.0 is what a single architect can build in 30 hours when discipline compounds. The trinity has one pillar complete. The next two pillars - Cop and Interpreter - will be built on the same substrate, under the same methodology, against the same doctrine corpus.

> *Every opcode declares its cost. Every grant declares its parent. Every doctrine declares its scope.*

Randolph Pelican III / StableTech Enterprises LLC
github.com/RandolphPelican/codebook
V1.0 SEAL contract sha: c9923b8cf9fb6caf4c195e2d0d0ea2ed4a8e51e4e9f827b1fc24dd0b28c1d900
May 2026